TP 2.1 - GENERADORES PSEUDOALEATORIOS

Valentino Laveggi
valentinolaveggi@gmail.com

Joaquín Tomás Murúa
jmurua03@gmail.com

Luca Olivieri
lucaolivieri03@gmail.com

Genaro Ortiz
genaro20038@gmail.com

Santiago Dedich
santidedich3226@gmail.com

Universidad Tecnológica Nacional - FRRO
Zeballos 1341, S2000, Argentina

May 31, 2026

ABSTRACT

El presente documento detalla el desarrollo, evaluación y análisis comparativo de algoritmos generadores de números pseudoaleatorios implementados en Python 3.x. Se analizan dos desarrollos algorítmicos propios: un Generador Congruencial Lineal (GCL) y el Método de los Cuadrados Medios, contrastándolos directamente con los generadores de uso profesional provistos por las bibliotecas nativas de Python y NumPy. Para validar las propiedades estadísticas fundamentales de uniformidad e independencia, se aplica una batería metodológica que incluye el Test de Chi-Cuadrado, el Test Monobit, la Prueba de Rachas y un análisis empírico visual mediante mapas de bits (Bitmap). Los resultados cuantitativos y cualitativos confirman la robustez del GCL y los generadores profesionales, los cuales superan exitosamente las pruebas con un nivel de significancia de $\alpha = 0.01$. Por el contrario, se evidencia el colapso estructural y la alta tasa de degeneración del Método de los Cuadrados Medios de forma empírica. Finalmente, se establecen conclusiones fundamentadas sobre la fiabilidad de estos algoritmos y el comportamiento estadístico esperado en el contexto de la simulación computacional moderna.

1 Introducción

En el campo de la simulación computacional, la capacidad de reproducir el comportamiento de variables aleatorias es un pilar fundamental. Para lograrlo, los sistemas informáticos recurren a la generación de números pseudoaleatorios. Un número pseudoaleatorio es el resultado de un proceso que aparenta producir valores al azar, pero que en realidad obedece a un algoritmo matemático completamente determinista. Lejos de ser una desventaja, esta característica es invaluable, ya que permite la replicabilidad exacta de las simulaciones para su posterior depuración y validación. Sin embargo, para que sean estadísticamente válidas, las secuencias generadas deben carecer de patrones visuales o regularidades, imitando con éxito las propiedades de uniformidad e independencia de una verdadera variable aleatoria.

El presente trabajo tiene por objetivo el desarrollo, análisis y contraste empírico de distintos algoritmos generadores de números pseudoaleatorios implementados en el lenguaje Python 3.x. Específicamente, se aborda la construcción de un **Generador Congruencial Lineal** (GCL) —como núcleo del estudio— y el histórico **Método de los Cuadrados Medios**. Para establecer un marco de referencia estadístico robusto, el rendimiento de estos desarrollos propios será contrastado directamente con dos algoritmos de uso profesional: la **función random nativa del lenguaje Python** y el generador de grado científico provisto por la **librería especializada NumPy**.

Para determinar rigurosamente la calidad de generación y garantizar que el GCL cumple con el comportamiento esperado, se someterán las secuencias a una batería metodológica compuesta por pruebas estadísticas y empíricas. En particular, se aplicarán el Test de **Chi-Cuadrado** (para evaluar la uniformidad de la distribución empírica), el **Test de**

Monobit (para verificar la equiprobabilidad a nivel binario), la **Prueba de Rachas** (orientada a validar la independencia de los datos) y un análisis visual de **Bitmap** (como prueba empírica de dispersión espacial).

Finalmente, como culminación del informe, se expondrán los resultados obtenidos mediante análisis tabulares y representaciones gráficas, estableciendo conclusiones fundamentadas sobre la viabilidad, robustez y las limitaciones inherentes de cada algoritmo estudiado.

2 Marco Teórico

2.1 Generadores de números aleatorios reales

Un generador de números aleatorios verdaderos (TRNG, por sus siglas en inglés) produce valores completamente impredecibles sin depender de un patrón matemático preestablecido. A grandes rasgos, los métodos de generación se dividen en dos categorías principales según su origen:

- **Hardware (Verdaderos o TRNG):** Utilizan la entropía de fenómenos físicos de naturaleza estocástica, tales como el ruido atmosférico, el decaimiento radiactivo o el ruido térmico de un procesador. Al ser genuinamente impredecibles, son el estándar ideal y necesario para aplicaciones de ciberseguridad y criptografía. Sin embargo, carecen de reproducibilidad, lo que los hace poco prácticos para la depuración y validación de modelos matemáticos.
- **Software (Pseudoaleatorios o PRNG):** Emplean algoritmos matemáticos rápidos y deterministas. Aunque no producen verdadera aleatoriedad física, su gran velocidad de cálculo y la capacidad de replicar exactamente la misma secuencia a partir de una semilla los hace perfectos para simulaciones computacionales, análisis estadístico y desarrollo de videojuegos.

2.2 Naturaleza de los Números Pseudoaleatorios

En la teoría de la simulación, existe una distinción fundamental entre los fenómenos puramente aleatorios (originados por procesos físicos impredecibles, como el ruido térmico) y los procesos pseudoaleatorios. Computacionalmente, resulta inviable generar verdadera aleatoriedad mediante la arquitectura de un procesador estándar. Por lo tanto, se recurre a los Generadores de Números Pseudoaleatorios (PRNG, por sus siglas en inglés).

Un PRNG es un algoritmo estrictamente determinista que, a partir de un valor inicial conocido como semilla (X_0), produce una secuencia de números que exhibe las mismas propiedades estadísticas que una secuencia verdaderamente aleatoria. Para que un PRNG sea considerado válido en el ámbito de la simulación, debe garantizar dos propiedades fundamentales:

- **Uniformidad:** Los valores generados, una vez normalizados al intervalo continuo $[0, 1)$, deben tener la misma probabilidad de ocurrencia. Se espera que los datos se ajusten a una distribución uniforme $U(0, 1)$.
- **Independencia:** La aparición de un valor específico en la secuencia no debe proveer ninguna información predictiva sobre el valor subsiguiente.

2.2.1 Generador Congruencial Lineal (GCL)

El Generador Congruencial Lineal (GCL), introducido originalmente por Derrick Lehmer en 1951, representa uno de los algoritmos más extendidos y computacionalmente eficientes para la generación de secuencias de números pseudoaleatorios. Fundamenta su comportamiento en una relación de recurrencia lineal sujeta a una operación de aritmética modular.

La relación de recurrencia para un Generador Congruencial Lineal se define formalmente como:

$$X_{n+1} = (aX_n + c) \cdot \quad (\text{mod } m) \tag{1}$$

Donde los parámetros y variables del sistema se restringen bajo las siguientes condiciones algebraicas:

- X_n es el valor entero actual de la secuencia ($0 \leq X_n < m$).
- X_0 corresponde a la semilla inicial del generador ($0 \leq X_0 < m$).
- a representa el multiplicador constante ($0 < a < m$).

- c define el incremento constante ($0 \leq c < m$). Cuando $c = 0$, el algoritmo se clasifica como un *Generador Multiplicativo de Lehmer*.
- m es el módulo del sistema ($m > 0$). Este parámetro determina el límite superior del espacio de estados y el período máximo potencial de la secuencia.

Para propósitos de simulación estocástica, los valores discretos generados se normalizan hacia el intervalo continuo $[0, 1)$ mediante la relación:

$$r_{n+1} = \frac{X_{n+1}}{m} \tag{2}$$

2.2.2 Método de los Cuadrados Medios

El método de los cuadrados medios es un algoritmo aritmético no lineal para la generación de números pseudoaleatorios. A partir de una semilla inicial X_0 de $2k$ dígitos, el algoritmo genera de forma recursiva el siguiente elemento de la secuencia mediante la elevación al cuadrado del valor anterior y la posterior extracción de los $2k$ dígitos centrales.

Matemáticamente, el proceso para obtener el siguiente número entero X_{i+1} se define mediante la ecuación:

$$X_{i+1} = \left\lfloor \frac{X_i^2}{10^k} \right\rfloor \cdot \pmod{10^{2k}} \tag{3}$$

Donde:

- X_i es el valor entero actual de la secuencia (cuya longitud es de $2k$ dígitos).
- $\lfloor x \rfloor$ representa la función piso (*floor*), encargada de redondear el valor x al entero menor o igual más cercano. En la fórmula, se utiliza para remover los k dígitos menos significativos de la derecha.
- $\pmod{10^{2k}}$ corresponde a la operación módulo que aísla los $2k$ dígitos centrales, truncando los dígitos excedentes de la izquierda.

Para normalizar el valor obtenido y transformarlo en un número pseudoaleatorio r_{i+1} en el intervalo estándar $[0, 1)$, se aplica la siguiente relación lineal:

$$r_{i+1} = \frac{X_{i+1}}{10^{2k}} \tag{4}$$

Nota metodológica: *Aunque el algoritmo posee relevancia histórica por ser el primer generador computacional, presenta limitaciones severas para la simulación moderna. Muestra una fuerte sensibilidad a la semilla inicial, una alta tasa de degeneración hacia el cero y una tendencia inherente a converger en ciclos cortos redundantes.*

2.2.3 Módulo `random` de Python (Mersenne Twister)

En el ámbito del desarrollo de software de propósito general, el lenguaje Python implementa de forma nativa la generación de pseudoaleatoriedad mediante el módulo estandar `random`. Este generador se fundamenta en el algoritmo *Mersenne Twister* (específicamente la variante MT19937), un generador de tipo congruencial generalizado por retroalimentación de registros de desplazamiento desarrollado por Makoto Matsumoto y Takuji Nishimura en 1997.

A nivel de diseño, el algoritmo opera sobre una matriz de estados de 19937 bits, lo que provee un período extraordinariamente extenso definido matemáticamente por un número primo de Mersenne:

$$P = 2^{19937} - 1 \tag{5}$$

El núcleo del proceso de generación se compone de dos fases principales:

1. **Transformación Lineal (*Twisting*):** Se computa una relación de recurrencia lineal matricial sobre vectores binarios que mezcla bits del estado actual para generar un nuevo vector de estado.
2. **Ajuste del Temperamento (*Tempering*):** El vector de estado intermedio se multiplica por una matriz invertible de tempering fija para mejorar la equidistribución estadística de los bits antes de retornar el número final.

Para obtener un valor normalizado r_n en el intervalo $[0, 1)$, el sistema extrae palabras de 53 bits de precisión para construir un punto flotante de doble precisión mediante la relación:

$$r_n = \frac{X_n}{2^{53}} \tag{6}$$

Nota metodológica: *Si bien MT19937 posee propiedades óptimas de equidistribución en hasta 623 dimensiones y un período casi ilimitado para simulaciones estadísticas estándar, carece de propiedades criptográficas. Debido a su linealidad estructural, observar una secuencia consecutiva de 624 valores de salida permite reconstruir algebraicamente el estado interno del generador y predecir con total certeza los valores futuros.*

2.2.4 Submódulo `numpy.random` (Algoritmo PCG64)

Para entornos de computación científica y simulación de alta velocidad, la biblioteca NumPy ha evolucionado su infraestructura de aleatoriedad (a partir de la versión 1.17) migrando del algoritmo Mersenne Twister hacia la familia de Generadores Congruenciales Permutados (PCG), empleando específicamente la arquitectura *PCG64* como motor estocástico predeterminado mediante la interfaz `default_rng()`.

El algoritmo PCG64, diseñado por Melissa O'Neill en 2014, mitiga las ineficiencias de los generadores tradicionales aplicando técnicas de aleatoriedad estadística sobre una base congruencial lineal. Su funcionamiento se divide en dos componentes desacoplados:

1. **Estado de Transición:** Utiliza un Generador Congruencial Lineal (GCL) de 128 bits de alta velocidad para actualizar el estado interno del sistema. Su periodo está acotado por 2^{128}.
2. **Función de Salida (Permutación):** En lugar de exponer el estado del GCL directamente, se aplica una función de permutación no lineal (*XSH-RR*: *eXclusive-or, Right shift, Random Rotate*) que trunca y transforma los 128 bits internos en una salida aleatoria de 64 bits de alta entropía.

Matemáticamente, la permutación destructiva elimina la linealidad del GCL subyacente, dificultando la predicción inversa del estado y mejorando drásticamente el rendimiento en pruebas de estrés estadístico como la suite *BigCrush*. La normalización a un número flotante $r_n \in [0, 1)$ se ejecuta dividiendo la salida transformada sobre el límite superior del espacio de 64 bits:

$$r_n = \frac{\text{XSH-RR}(X_n)}{2^{64}} \tag{7}$$

Nota de diseño: *PCG64 supera a Mersenne Twister en eficiencia computacional, huella de memoria (emplea solo 128 bits de estado frente a los más de 2.4 KB de MT19937) y robustez en entornos multihilo, impidiendo la correlación serial entre flujos paralelos de simulación.*

3 Metodología de Evaluación

Para que un generador de números pseudoaleatorios (PRNG) sea considerado matemáticamente apto para la simulación de sistemas, la secuencia de valores que produce debe someterse a un riguroso escrutinio estadístico. En el ámbito industrial y criptográfico, la evaluación de estos generadores se rige por baterías estandarizadas, siendo la referencia internacional más adoptada la suite *NIST SP 800-22* (A Statistical Test Suite for Random and Pseudorandom Number Generators).

Atendiendo a los objetivos de este trabajo práctico, se ha seleccionado una batería representativa y reducida de cuatro pruebas (inspiradas en dichos estándares) orientadas a verificar empíricamente las dos propiedades estocásticas fundamentales: la **uniformidad** (garantizando que todos los valores en el intervalo $[0, 1)$ tienen la misma probabilidad de ocurrencia) y la **independencia** (asegurando que ningún valor puede predecirse a partir de sus predecesores).

Para todas las evaluaciones numéricas de este trabajo, se ha establecido un nivel de significancia $\alpha = 0,01$. A continuación, se detallan las metodologías implementadas.

3.1 Test de Chi-Cuadrado — *Prueba de Uniformidad*

Esta prueba de bondad de ajuste evalúa empíricamente si la distribución de los números generados se desvía de forma significativa de la distribución uniforme continua esperada. El procedimiento consiste en dividir el intervalo $[0, 1)$ en

k subintervalos idénticos. Posteriormente, se contabiliza la cantidad de valores que caen dentro de cada subintervalo (frecuencia observada, O_i) y se contrasta empíricamente con la cantidad de valores que matemáticamente deberían caer en dicho rango (frecuencia esperada, $E_i = n/k$).

El estadístico de prueba se calcula mediante la siguiente sumatoria:

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i} \tag{8}$$

Un valor de χ^2 elevado indica una asimetría en la distribución. El estadístico se contrasta contra una distribución Chi-cuadrado con $k - 1$ grados de libertad para obtener el p-valor. Si el p-valor $> \alpha$, se concluye que no hay evidencia estadística para rechazar la uniformidad de la secuencia.

3.2 Test Monobit (Frecuencia) — *Uniformidad Binaria Global*

Propuesto dentro del estándar SP 800-22 del NIST, este test analiza la proporción global de ceros y unos en toda la secuencia para garantizar un equilibrio equiprobable. Para su implementación, la secuencia continua se binariza: cada número generado se convierte en un bit, asignándole el valor $+1$ si es estrictamente mayor a $0,5$, y el valor -1 si es menor o igual a $0,5$.

Si el generador es perfectamente uniforme, la suma absoluta de todos los bits (S_n) debería gravitar fuertemente hacia el cero. El cálculo de significancia estadística se realiza aplicando la función de error complementaria (erfc):

$$p\text{-valor} = \mathrm{erfc}\left(\frac{|S_n|}{\sqrt{n \cdot 2}}\right) \quad \text{donde} \quad S_n = \sum_{i=1}^{n} b_i \tag{9}$$

Un p-valor $> \alpha$ indica que la proporción espacial entre valores altos y bajos es estadísticamente equilibrada y admisible.

3.3 Test de Rachas (*Runs Test*) — *Prueba de Independencia*

Esta prueba paramétrica tiene como objetivo fundamental detectar correlaciones lineales y no lineales entre valores adyacentes de la secuencia. Se define una *racha* como una sucesión ininterrumpida de valores que se encuentran todos por encima o todos por debajo de la mediana del conjunto de datos.

Si el generador es genuinamente independiente, la oscilación de la secuencia sobre la mediana generará un número total de rachas (R) que se ajusta a una variable aleatoria normal. Si n_1 representa la cantidad de valores superiores a la mediana y n_2 los inferiores, la media poblacional esperada (μ_R) y la varianza (σ_R^2) de las rachas se definen como:

$$\mu_R = \frac{2n_1n_2}{n_1 + n_2} + 1 \tag{10}$$

$$\sigma_R^2 = \frac{2n_1n_2(2n_1n_2 - n_1 - n_2)}{(n_1 + n_2)^2(n_1 + n_2 - 1)} \tag{11}$$

A partir de estos parámetros, se estandariza el estadístico para obtener el valor Z:

$$Z = \frac{R - \mu_R}{\sqrt{\sigma_R^2}} \tag{12}$$

Un p-valor bilateral derivado de Z superior a α ratifica la independencia serial del modelo.

3.4 Test de Bitmap — *Inspección Empírica Bidimensional*

A diferencia de los modelos estadísticos anteriores, el análisis mediante mapas de bits constituye una evaluación heurística y cualitativa. Consiste en proyectar la secuencia pseudoaleatoria unidimensional sobre una matriz bidimensional (grilla de píxeles). Al igual que en el test Monobit, la secuencia se discretiza: los valores $> 0,5$ se grafican como píxeles blancos y los valores $\leq 0,5$ como píxeles negros.

Esta técnica de renderizado permite aprovechar el excepcional reconocimiento de patrones del sistema visual humano. Una secuencia validada empíricamente presentará el aspecto de ruido blanco estático (ausencia de información estructurada), mientras que los generadores defectuosos revelarán bandas, diagonales, retículas o gradientes manifiestos.

4 Implementación utilizada para el estudio

Con el objetivo de contrastar empíricamente las propiedades estadísticas de los cuatro generadores analizados, se implementó una batería de pruebas en Python 3.x aplicada sobre muestras de $n = 10.000$ valores generados en el intervalo $[0, 1)$. El nivel de significancia adoptado fue $\alpha = 0{,}01$ para la totalidad de las evaluaciones. En cuanto a la aplicación de los Generadores, utilizamos Numpy, Random de Python y el desarrollo matemático y la parametrización sesgada para los métododos de GCL y Medios Cuadrados.

El **Test de Chi-Cuadrado** se utilizó para verificar la uniformidad de la distribución empírica de cada generador. Los 10.000 valores se clasificaron en $k = 10$ subintervalos de igual amplitud y se contrastaron las frecuencias observadas contra la frecuencia teórica esperada de 1.000 valores por intervalo. Un p-valor superior a α indica que la secuencia no presenta desvíos significativos respecto de la distribución uniforme.

El **Test Monobit** se empleó como verificación global de equiprobabilidad. Cada valor generado fue binarizado según su posición relativa al umbral $0{,}5$, y se evaluó si la cantidad de valores en cada mitad del intervalo resultaba estadísticamente equilibrada. Esta prueba permite detectar sesgos globales que el Chi-Cuadrado, al trabajar con intervalos más granulares, podría no capturar con igual sensibilidad.

La **Prueba de Rachas** se aplicó para evaluar la independencia serial de las secuencias. Se contabilizó la cantidad de rachas respecto de la mediana de cada muestra y se comparó contra el valor esperado bajo la hipótesis de independencia. Un Z-score cercano a cero y un p-valor holgado indican que la secuencia oscila de forma natural alrededor de la mediana, sin tendencias ni ciclos detectables.

Finalmente, el **análisis de Bitmap** se utilizó como herramienta cualitativa complementaria. Cada secuencia fue proyectada sobre una grilla bidimensional de píxeles binarios, permitiendo detectar visualmente estructuras, bandas o agrupamientos que los tests numéricos podrían no capturar. Una imagen con apariencia de ruido estático homogéneo valida empíricamente la ausencia de patrones en el generador evaluado. Este en particular fue el más gráfico a la hora de mostrar la disconformidad con el Generador de Medios Cuadrados y su sensibilidad ante semillas poco convencionales.

La implementación de los tests al igual que los métodos GCL y Medios Cuadrados implican un desarrollo directo en la funcionalidad del código.

5 Exposición de los resultados y análisis

En esta sección se exponen y contrastan los resultados empíricos obtenidos tras someter a los cuatro generadores seleccionados en base a la metodologia utilizada.

5.1 Comparativa y Análisis Estadístico de los Generadores

Los análisis numéricos detallados en las tablas a continuación permiten evaluar de forma cuantitativa el cumplimiento de las hipótesis de uniformidad e independencia estadística.

5.1.1 Análisis del Método de los Cuadrados Medios

Como se observa de forma contundente en la Tabla 2, el método de los Cuadrados Medios sufrió un colapso estadístico absoluto en las tres pruebas realizadas, obteniendo en todos los casos un p-valor $= 0,0000$.

Este fenómeno es una manifestación matemática directa de las limitaciones intrínsecas del algoritmo. El estadístico de Chi-Cuadrado extremadamente elevado ($Stat = 89261,5820$) y el resultado del Test Monobit ($Suma = |9978|$) demuestran que el generador entró rápidamente en un bucle de degeneración o punto fijo (comúnmente una convergencia prematura hacia el cero absoluto o hacia secuencias repetitivas monótonas). Al quedar atrapado en este ciclo corto, casi la totalidad de los 10.000 valores cayeron en una misma región del intervalo, anulando por completo la uniformidad y provocando un fallo masivo en el test de rachas ($Z = -99,4592$). Esto ratifica de forma empírica por qué este método histórico se considera inviable para simulaciones contemporáneas.

5.1.2 Análisis del Generador Congruencial Lineal (GCL)

A diferencia del método anterior, el GCL desarrollado de forma propia (Tabla 1) exhibió un comportamiento sumamente robusto, superando con éxito todas las pruebas estadísticas para el nivel $\alpha = 0,01$ establecido.

El test de Chi-Cuadrado arrojó una desviación muy baja ($Stat = 6,6200$) con un p-valor $= 0,6766$, lo que denota una excelente distribución uniforme a lo largo de los diez intervalos bajo análisis. Asimismo, el test de Monobit confirma

una correcta equiprobabilidad en la dispersión global (p-valor $= 0,6171$). Con respecto a la independencia, la prueba de rachas devolvió un p-valor $= 0,0601$. Aunque este último valor se encuentra relativamente cercano a la zona crítica de rechazo, resulta perfectamente válido y estadísticamente aceptable, evidenciando únicamente la ligera tendencia a la correlación serial de corto alcance que es característica en las estructuras algebraicas lineales de tipo congruencial.

5.1.3 Análisis de los Generadores Profesionales (Python y NumPy)

Los generadores de referencia incluidos para el contraste (Tablas 3 y 4) ratificaron las propiedades estadísticas esperadas para algoritmos de producción masiva.

El módulo `random` nativo de Python mostró el perfil de uniformidad más elevado del estudio, con un p-valor $= 0,7515$ en la prueba de Chi-Cuadrado. Por otra parte, el generador de grado científico provisto por `NumPy` se destacó de manera excepcional en la métrica de independencia; su prueba de rachas arrojó un score normalizado prácticamente nulo ($Z = -0,2400$) y un p-valor $= 0,8103$. Este elevado indicador de independencia lineal y no lineal confirma la idoneidad de la librería para simulaciones de alta complejidad multidimensional, donde se requiere mitigar al máximo cualquier patrón de vecindad entre muestras sucesivas.

5.2 Análisis Cualitativo mediante Inspección Visual (Bitmap)

Para robustecer las conclusiones cuantitativas, las secuencias de datos se proyectaron en un plano bidimensional en forma de mapas de bits de grilla binaria.

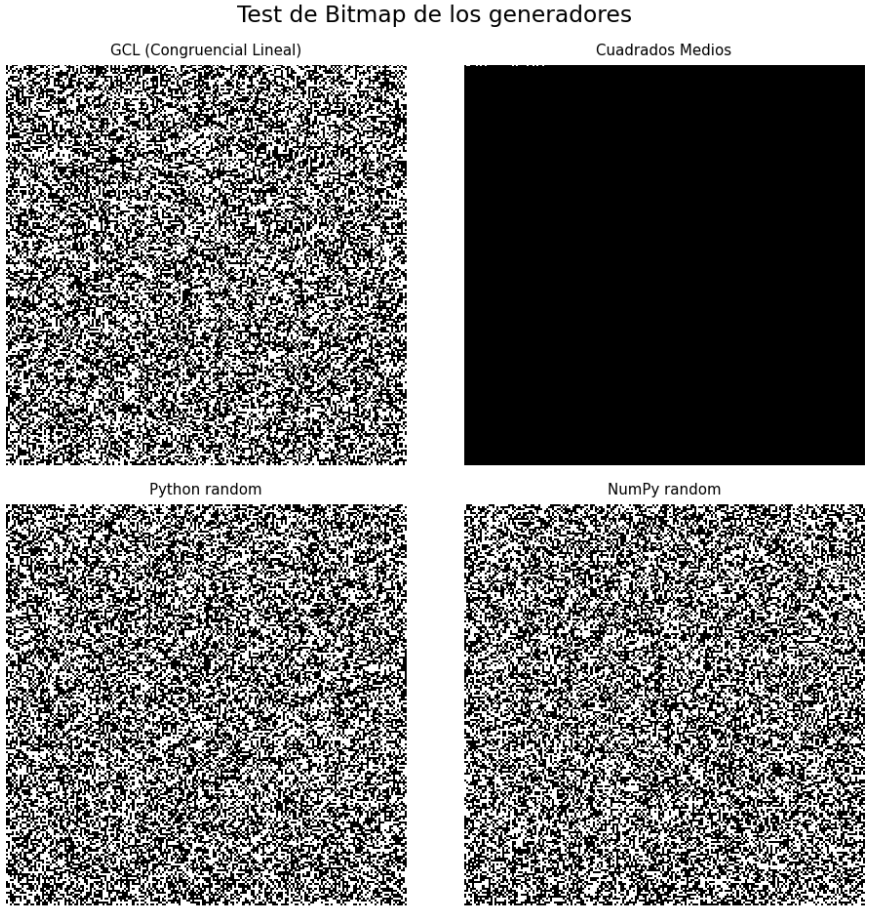


Figure 1: Resultados del mapa de bits de los 4 generadores de números pseudoaleatorios.

La inspección visual de la Figura 1 guarda una correlación directa con los estadísticos analizados. Los mapas correspondientes al GCL, Python `random` y NumPy exhiben una textura homogénea tridimensional equivalente al ruido

blanco estático, validando visualmente la ausencia de clústeres espaciales o densidades anómalas. En contraposición, el mapa de bits asociado al método de Cuadrados Medios revela un patrón completamente plano o fuertemente estructurado, lo que expone visualmente la pérdida total de entropía y el estancamiento cíclico diagnosticado previamente en los tests numéricos.

5.3 Tablas de Resultados Detalladas

5.3.1 Tabla GCL

Prueba Estadística	**Estadístico de Prueba**	**p-value**	**Resultado** ($\alpha = 0.01$)
Chi-Cuadrado (Uniformidad)	Stat = 6.6200	0.6766	PASA
Monobit (Frecuencia)	Suma = $\|50\|$	0.6171	PASA
Rachas (Independencia)	Z-score = -1.8801	0.0601	PASA

Table 1: Resultados de las pruebas para el Generador Congruencial Lineal (GCL)

5.3.2 Tabla Cuadrados Medios

Prueba Estadística	**Estadístico de Prueba**	**p-value**	**Resultado** ($\alpha = 0.01$)
Chi-Cuadrado (Uniformidad)	Stat = 89261.5820	0.0000	FALLA
Monobit (Frecuencia)	Suma = $\|9978\|$	0.0000	FALLA
Rachas (Independencia)	Z-score = -99.4592	0.0000	FALLA

Table 2: Resultados de las pruebas para el Método de los Cuadrados Medios

5.3.3 Tabla Python random

Prueba Estadística	**Estadístico de Prueba**	**p-value**	**Resultado** ($\alpha = 0.01$)
Chi-Cuadrado (Uniformidad)	Stat = 5.8840	0.7515	PASA
Monobit (Frecuencia)	Suma = $\|62\|$	0.5353	PASA
Rachas (Independencia)	Z-score = -1.9001	0.0574	PASA

Table 3: Resultados de las pruebas para el generador nativo de Python (random)

5.3.4 Tabla NumPy random

Prueba Estadística	**Estadístico de Prueba**	**p-value**	**Resultado** ($\alpha = 0.01$)
Chi-Cuadrado (Uniformidad)	Stat = 8.9740	0.4397	PASA
Monobit (Frecuencia)	Suma = $\|44\|$	0.6599	PASA
Rachas (Independencia)	Z-score = -0.2400	0.8103	PASA

Table 4: Resultados de las pruebas para el generador científico NumPy (numpy.random)

6 Conclusiones

El presente trabajo práctico permitió contrastar de forma empírica la base teórica de la generación de números pseudoaleatorios con su comportamiento computacional real. A partir de los resultados obtenidos tras someter a los algoritmos a la batería de pruebas estadísticas (Chi-Cuadrado, Monobit, Rachas) y la inspección visual (Bitmap), se pueden establecer las siguientes conclusiones:

En primer lugar, se corroboró de forma contundente la obsolescencia del **Método de los Cuadrados Medios** para aplicaciones prácticas modernas. Tal como predecía la teoría, el comportamiento empírico del algoritmo demostró una

alta inestabilidad y una rápida convergencia hacia la degeneración (bucles cíclicos repetitivos). El fallo sistemático en todas las pruebas de uniformidad e independencia (p-valor $= 0,0000$) y la clara estructuración en su mapa de bits confirman que no es una herramienta confiable para la simulación de sistemas estocásticos.

Por el contrario, el **Generador Congruencial Lineal (GCL)** desarrollado cumplió satisfactoriamente con las expectativas. Logró superar el umbral de significancia ($\alpha = 0,01$) en los test de uniformidad y mantuvo una independencia estadística aceptable. Esto demuestra que, mediante una correcta elección de sus parámetros (módulo, multiplicador e incremento), una estructura matemática simple y determinista es capaz de emular con éxito el comportamiento del azar continuo. Sin embargo, su proximidad al límite de rechazo en el test de rachas recuerda que los GCL poseen limitaciones inherentes, lo que los hace vulnerables en simulaciones de extrema dimensionalidad.

Al comparar nuestros desarrollos con los generadores de estándar industrial (**Python** `random` y **NumPy**), se evidenció la superioridad de estos últimos, especialmente en el rubro de la independencia lineal (Test de Rachas). Algoritmos avanzados como el *Mersenne Twister* (utilizado por el módulo nativo de Python) están diseñados para poseer períodos astronómicos y evitar cualquier tipo de correlación serial a corto o largo plazo, justificando su uso profesional.

Finalmente, una conclusión fundamental que deja este análisis es comprender la naturaleza de la evaluación estadística: paradójicamente, si un generador de números pseudoaleatorios arrojara resultados perfectos en el 100% de las pruebas todo el tiempo, debería generar sospechas. Un generador (o fenómeno) verdaderamente aleatorio ocasionalmente producirá secuencias que parecen "no aleatorias" debido a fluctuaciones estadísticas normales. La clave empírica de un buen simulador de aleatoriedad radica en que los desvíos ocurran estrictamente dentro de los márgenes de probabilidad y varianza esperados, sin mostrar patrones sistemáticos a lo largo del tiempo.

7 Referencias

References

[1] RANDOM.ORG, *Statistical Analysis*. Disponible en: `https://www.random.org/analysis/`

[2] Kenny, C. (2005). *Analysis of the RANDOM.ORG Random Number Generator*. Trinity College, Dublin. Disponible en: `https://www.random.org/analysis/Analysis2005.pdf`

[3] Numeros Pseudoaleatorios. *Estudio de Numeros Pseudoaleatorios*. Disponible en: `https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html`

[4] Python Software Foundation. *random — Generar números pseudoaleatorios*. Disponible en: `https://docs.python.org/es/3.9/library/random.html`

[5] NumPy Developers. *numpy.random.rand — NumPy Manual*. Disponible en: `https://numpy.org/doc/stable/reference/random/generated/numpy.random.rand.html`